Driver Drowsiness Detection System

Complete Code Explanation — Block by Block with Input & Output

Color Legend			
Blue box = What We Did	Purple box = Input	Green box = Output	Code box = Code

Block 1 — Mount Google Drive

Code:

```
from google.colab import drive
drive.mount('/content/drive')
```

What We Did:

We imported the drive module from Google Colab's library and called mount() to connect our Google Drive account to the Colab environment at the path /content/drive. Without this step, Colab cannot access any files stored in our personal Drive — including our dataset ZIP file. This is always the very first step in any Colab project that depends on Drive-stored files.

Input:

Google account authorization — a popup appears asking the user to sign in and grant Colab access to Google Drive.

Output:

Printed on screen: 'Mounted at /content/drive' — confirming Drive is now accessible as a local folder and all files inside it can be read, written, or executed within the session.

Block 2 — Extract ZIP File and Verify Folder Structure

Code:

```
import zipfile, os
zip_path = '/content/drive/MyDrive/drowsiness_project/archive (2).zip'
extract_path = '/content/dataset'
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
zip_ref.extractall(extract_path)
for root, dirs, files in os.walk(extract_path):
print indented folder names at each level
```

What We Did:

We used Python's built-in zipfile module to open the compressed dataset archive stored in Google Drive and extracted all its contents into /content/dataset — which is local Colab storage. The os.walk() function then

recursively traversed the extracted directory tree, printing each folder name with indentation proportional to its depth level so we could visually confirm the complete structure was correctly unpacked.

Input:

ZIP archive file: archive (2).zip located at /content/drive/MyDrive/drowsiness_project/. This file contains all dataset images organized in class subfolders.

Output:

Printed: 'Extracting ZIP file - please wait...' then 'Extraction done!' followed by an indented folder tree showing: dataset/ containing train/ which contains Closed/, Open/, no_yawn/, and yawn/ subfolders — confirming all four class image folders are intact and ready for use.

Block 3 — Count Images Per Class and Plot Distribution

Code:

```
classes = ['Closed', 'Open', 'no_yawn', 'yawn']
for cls in classes:
count = len(os.listdir(os.path.join(dataset_path, cls)))
class_counts[cls] = count
plt.bar(class_counts.keys(), class_counts.values(),
color=['red', 'green', 'blue', 'orange'])
plt.title('Dataset Class Distribution')
plt.show()
```

What We Did:

For each of the 4 class folders inside the training directory, we listed all files using os.listdir() and counted them with len() to find how many images belong to each class. The counts were stored in a dictionary and plotted as a bar chart using matplotlib to visually confirm the dataset is balanced. A balanced dataset is critical because if one class has far more images, the model will be biased toward predicting that class more often.

Input:

The four class subdirectories: Closed/, Open/, no_yawn/, yawn/ inside /content/dataset/train.

Output:

Printed per-class image count (approximately 725 images per class). A color bar chart displayed with four equal-height bars in red (Closed), green (Open), blue (no_yawn), and orange (yawn) — visually confirming the dataset is evenly balanced across all 4 classes with no class imbalance issue.

Block 4 — Display Sample Images from Each Class

Code:

```
fig, axes = plt.subplots(2, 4, figsize=(15, 8))
for idx, cls in enumerate(classes):
for row in range(2):
```

```
img_path = os.path.join(cls_path, random.choice(images))
img = mpimg.imread(img_path)
axes[row][idx].imshow(img, cmap='gray')
axes[row][idx].set_title(cls)
axes[row][idx].axis('off')
plt.show()
```

What We Did:

We created a 2-row by 4-column subplot grid using matplotlib. For each class, we randomly selected 2 image filenames using random.choice(), read the image from disk using mpimg.imread(), and displayed it in the corresponding grid cell with its class name as the title. The axis was turned off to remove tick marks and numbers. This is an exploratory data analysis step to visually confirm image quality, verify correct class labeling, and understand the visual differences between eye-open, eye-closed, yawning, and non-yawning images before building the model.

Input:

Randomly selected image files from each of the 4 class folders — 2 images per class, 8 images total, chosen at random from all available images in each class directory.

Output:

A 2x4 image grid displayed on screen showing 8 sample grayscale facial images — 2 images each for Closed eyes, Open eyes, no_yawn mouth, and yawn mouth — labeled by class name, confirming the images are clear, distinguishable, and correctly organized.

Block 5 — First Data Generator Setup Using Original Folders

Code:

```
train_datagen = ImageDataGenerator(
rescale=1./255, rotation_range=20, zoom_range=0.2,
horizontal_flip=True, brightness_range=[0.8, 1.2])
val_datagen = ImageDataGenerator(rescale=1./255)
test_datagen = ImageDataGenerator(rescale=1./255)
train_generator = train_datagen.flow_from_directory(
'/content/dataset/train', target_size=(224,224),
batch_size=32, class_mode='categorical')
```

What We Did:

We configured TensorFlow's ImageDataGenerator for all three splits. Training data gets five augmentation techniques: rescaling pixels from 0-255 to 0-1 (normalization), random rotation up to 20 degrees, zoom in/out up to 20%, random horizontal flipping, and brightness variation between 80% and 120% of original — all to artificially increase training variety and help the model generalize to real-world conditions. Validation and test generators only apply rescaling without augmentation because their purpose is to accurately measure real-world performance. All images are resized to 224x224 pixels to match model input requirements.

Input:

Original dataset folder paths: /content/dataset/train, /content/dataset/valid, /content/dataset/test with images in 4 class subfolders each.

Output:

Printed for each split: 'Found X images belonging to 4 classes.' Also printed class indices dictionary: {'Closed': 0, 'Open': 1, 'no_yawn': 2, 'yawn': 3} confirming correct label encoding. Generators are now ready to feed batches of 32 augmented images to the model.

Block 6 — Verify Folder Contents

Code:

```
for item in os.listdir('/content/dataset'):
print(item)
for item in os.listdir('/content/dataset/train'):
print(item)
```

What We Did:

We used os.listdir() to print the names of all items in the top-level dataset directory and then separately listed all items inside the train subdirectory. This quick verification step was done before the manual split to confirm the dataset structure was intact, that all four class folders existed inside train, and that no folders were missing or misnamed — which would cause errors in the splitting code.

Input:

Directory paths: /content/dataset and /content/dataset/train.

Output:

Printed top-level folders: train/, valid/, test/ confirming the three splits exist. Printed train subfolders: Closed/, Open/, no_yawn/, yawn/ confirming all 4 class folders are present with correct names before proceeding to the manual splitting step.

Block 7 — Manual Dataset Split — 70% Train / 15% Validation / 15% Test

Code:

```
for cls in classes:
images = os.listdir(cls_path)
random.shuffle(images)
train_end = int(0.70 * total)
val_end = int(0.85 * total)
train_imgs = images[:train_end]
val_imgs = images[train_end:val_end]
test_imgs = images[val_end:]
for img in train_imgs:
shutil.copy(src_path, train_dest_path)
```

```
# same for val and test
```

What We Did:

Since the original dataset had no proper train/val/test split, we built one manually. For each class independently: all image filenames were listed, randomly shuffled using random.shuffle() to eliminate any ordering bias, and then sliced into three segments using calculated index boundaries — first 70% for training, next 15% for validation, final 15% for testing. Each image was physically copied using shutil.copy() into the correct class subfolder inside newly created split_dataset/train, split_dataset/valid, and split_dataset/test directories. Doing this per class ensures the same 70/15/15 ratio is maintained for every class individually, avoiding any class imbalance in the splits.

Input:

All images from /content/dataset/train across all 4 classes — approximately 725 images per class, 2900 images total.

Output:

Printed per-class split confirmation, e.g., 'Closed: Train=507, Val=109, Test=109' for each of the 4 classes. 'Dataset split done!' printed at the end. New split_dataset/ directory created with train/valid/test subfolders each containing Closed/, Open/, no_yawn/, yawn/ with the correctly distributed images copied in.

Block 8 — Final Data Generators Using Split Dataset

Code:

```
train_generator = train_datagen.flow_from_directory(
'/content/split_dataset/train', target_size=(224,224),
batch_size=32, class_mode='categorical')
val_generator = val_datagen.flow_from_directory(
'/content/split_dataset/valid', ...)
test_generator = test_datagen.flow_from_directory(
'/content/split_dataset/test', ..., shuffle=False)
```

What We Did:

We recreated all three data generators now pointing to the correctly split dataset folders. The same augmentation configuration from Block 5 was applied. shuffle=False was set on the test generator so predictions remain in the same order as the actual labels — essential for accurate confusion matrix and classification report generation. These are the final generators used for all subsequent model training and evaluation steps.

Input:

split_dataset/train (2029 images), split_dataset/valid (435 images), split_dataset/test (436 images) — all with 4 class subfolders, all correctly split from Block 7.

Output:

Printed class indices: {'Closed':0, 'Open':1, 'no_yawn':2, 'yawn':3}. Printed exact sample counts: Train samples: 2029, Validation samples: 435, Test samples: 436. Generators are confirmed ready to supply shuffled augmented batches for training and ordered clean batches for evaluation.

Block 9 — Build Custom CNN Model Architecture

Code:

```
cnn_model = Sequential([
Conv2D(32,(3,3),activation='relu',input_shape=(224,224,3)),
BatchNormalization(), MaxPooling2D(2,2),
Conv2D(64,(3,3),activation='relu'),
BatchNormalization(), MaxPooling2D(2,2),
Conv2D(128,(3,3),activation='relu'),
BatchNormalization(), MaxPooling2D(2,2),
Conv2D(256,(3,3),activation='relu'),
BatchNormalization(), MaxPooling2D(2,2),
Flatten(),
Dense(512, activation='relu'), Dropout(0.5),
Dense(256, activation='relu'), Dropout(0.3),
Dense(4, activation='softmax')
])
cnn_model.compile(optimizer='adam',
loss='categorical_crossentropy', metrics=['accuracy'])
```

What We Did:

We constructed a deep CNN from scratch using Keras Sequential API. Four convolutional blocks progressively learn visual features: Block 1 (32 filters) detects basic edges and gradients, Block 2 (64 filters) learns simple shapes, Block 3 (128 filters) captures complex textures, Block 4 (256 filters) recognizes high-level facial features like eye shapes and mouth states. BatchNormalization after each Conv2D normalizes activations to stabilize training and reduce internal covariate shift. MaxPooling2D halves spatial dimensions each time, reducing memory and computation while retaining key features. Flatten() converts the 3D feature map to a 1D vector. Two Dense layers with Dropout (0.5 and 0.3) perform classification while preventing overfitting. The final softmax layer outputs 4 probability values — one per class — that sum to 1.0. Adam optimizer adapts the learning rate automatically. Categorical cross-entropy measures the error for multi-class classification.

Input:

Image tensors of shape (224, 224, 3) — 224 pixels height, 224 pixels width, 3 RGB color channels — fed in batches of 32.

Output:

Full model summary printed showing every layer name, output shape after each layer, and number of trainable parameters. Total trainable parameters approximately 8-10 million. Model is compiled and ready for training.

Block 10 — Train Custom CNN Model

Code:

```
early_stop = EarlyStopping(monitor='val_accuracy',
patience=5, restore_best_weights=True)
checkpoint = ModelCheckpoint('/content/cnn_best_model.h5',
monitor='val_accuracy', save_best_only=True, verbose=1)
cnn_history = cnn_model.fit(
train_generator, epochs=15,
validation_data=val_generator,
callbacks=[early_stop, checkpoint])
```

What We Did:

We trained the CNN using fit() for a maximum of 15 epochs. Each epoch processes all 2029 training images in batches of 32, computes predictions, calculates loss using categorical cross-entropy, and updates weights using Adam backpropagation. After each epoch, the model is evaluated on 435 validation images. EarlyStopping monitors validation accuracy and stops training if it does not improve for 5 consecutive epochs, then restores the best weights seen — this prevents wasting compute time and avoids overfitting. ModelCheckpoint saves the model to disk only when validation accuracy improves, ensuring the saved file always contains the best-performing version.

Input:

train_generator supplying 2029 augmented training images in batches of 32. val_generator supplying 435 clean validation images for evaluation after each epoch.

Output:

Epoch-by-epoch log printed showing: Epoch number, loss, accuracy, val_loss, val_accuracy for each of the completed epochs. Model saved automatically when val_accuracy improves (printed: 'val_accuracy improved... saving model'). Final line: 'CNN Training Complete!' and 'Best Validation Accuracy: 0.XXXX'. Best model file saved as cnn_best_model.h5.

Block 11 — Plot CNN Training Accuracy and Loss Graphs

Code:

```
plt.subplot(1,2,1)
plt.plot(cnn_history.history['accuracy'], label='Train Accuracy')
plt.plot(cnn_history.history['val_accuracy'], label='Val Accuracy')
plt.title('CNN Model - Accuracy')
plt.subplot(1,2,2)
plt.plot(cnn_history.history['loss'], label='Train Loss')
plt.plot(cnn_history.history['val_loss'], label='Val Loss')
plt.title('CNN Model - Loss')
plt.show()
```

What We Did:

We extracted accuracy and loss values recorded during training from the cnn_history object and plotted them as two side-by-side line graphs. The accuracy graph shows both training and validation accuracy across all epochs —

if validation accuracy is much lower than training accuracy, the model is overfitting. The loss graph shows both training and validation loss — both should decrease over time for a well-training model. These graphs are essential diagnostic tools to understand model learning behavior.

Input:

cnn_history.history dictionary containing lists of accuracy, val_accuracy, loss, and val_loss values — one value per completed training epoch.

Output:

Two matplotlib line charts displayed side by side: Left chart shows training accuracy (blue line) and validation accuracy (orange line) rising across epochs. Right chart shows training loss (blue) and validation loss (orange) decreasing across epochs. Both confirm the CNN was learning but with some gap between training and validation curves.

Block 12 — Build MobileNetV2 Transfer Learning Model

Code:

```
base_model = MobileNetV2(input_shape=(224,224,3),
include_top=False, weights='imagenet')
base_model.trainable = False
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.3)(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.2)(x)
output = Dense(4, activation='softmax')(x)
mobilenet_model = Model(inputs=base_model.input, outputs=output)
mobilenet_model.compile(optimizer='adam',
loss='categorical_crossentropy', metrics=['accuracy'])
```

What We Did:

We loaded MobileNetV2 pretrained on ImageNet (1 million+ images, 1000 classes) using include_top=False to remove its original classification head, keeping only the powerful feature extraction layers. Setting base_model.trainable=False froze all 154 base layers so their valuable pretrained weights were preserved — the model already knows how to detect edges, textures, and shapes from ImageNet. We added a custom head: GlobalAveragePooling2D compressed the 7x7x1280 feature map to a 1280-dimensional vector, then two Dense layers with Dropout refined the representation for our specific task, and a 4-neuron softmax output produced class probabilities. Only the custom top layers are trainable, making this much faster and more efficient than training from scratch.

Input:

Pretrained MobileNetV2 weights downloaded automatically from TensorFlow's model repository (ImageNet weights). Input image tensors of shape (224, 224, 3).

Output:

Printed: 'MobileNetV2 model built!' Total layer count (~156 layers) and total trainable parameter count printed. Only approximately 400,000 parameters are trainable (the custom top layers) while the 3+ million base parameters are frozen — confirming transfer learning setup is correct.

Block 13 — Train MobileNetV2 Model

Code:

```
early_stop = EarlyStopping(monitor='val_accuracy',
patience=5, restore_best_weights=True)
checkpoint = ModelCheckpoint(
'/content/mobilenet_best_model.keras',
monitor='val_accuracy', save_best_only=True, verbose=1)
mobilenet_history = mobilenet_model.fit(
train_generator, epochs=15,
validation_data=val_generator,
callbacks=[early_stop, checkpoint])
```

What We Did:

We trained MobileNetV2 using the same approach as the CNN — fit() for up to 15 epochs with EarlyStopping and ModelCheckpoint callbacks. Because the base MobileNetV2 layers are frozen, backpropagation only updates the small custom top layers during each epoch. This means each epoch trains much faster than the CNN. The pretrained feature extraction layers already produce rich, meaningful features, so the custom layers converge quickly to correct classification. The best model is saved in the newer .keras format.

Input:

train_generator with 2029 augmented training images in batches of 32. val_generator with 435 validation images for per-epoch evaluation.

Output:

Epoch-by-epoch training log showing loss, accuracy, val_loss, val_accuracy for each epoch. Model saved whenever val_accuracy improves. Final printed result: 'MobileNetV2 Training Complete!' and 'Best Validation Accuracy: 0.9220' — confirming 92.20% validation accuracy achieved. Best model saved as mobilenet_best_model.keras.

Block 14 — Plot MobileNetV2 Training Graphs

Code:

```
plt.plot(mobilenet_history.history['accuracy'], label='Train Accuracy')
plt.plot(mobilenet_history.history['val_accuracy'], label='Val Accuracy')
plt.plot(mobilenet_history.history['loss'], label='Train Loss')
plt.plot(mobilenet_history.history['val_loss'], label='Val Loss')
plt.show()
```

What We Did:

Same visualization approach as Block 11 but applied to MobileNetV2 training history. Plotting both models' training curves allows direct comparison of learning behavior — we can see how quickly MobileNetV2 converges compared to the CNN, whether it overfits less or more, and how stable validation accuracy is across epochs.

Input:

mobilenet_history.history dictionary with epoch-wise accuracy, val_accuracy, loss, and val_loss lists.

Output:

Two matplotlib line charts showing MobileNetV2's training vs validation accuracy and loss across all epochs. The curves show steeper early improvement and higher final accuracy compared to the CNN graphs, visually demonstrating the advantage of transfer learning — strong performance achieved with fewer epochs.

Block 15 — Evaluate Both Models — Confusion Matrix and Final Comparison

Code:

```
cnn_test_loss, cnn_test_acc = cnn_model.evaluate(test_generator)
cnn_preds = cnn_model.predict(test_generator)
cnn_pred_classes = np.argmax(cnn_preds, axis=1)
true_classes = test_generator.classes
cm_cnn = confusion_matrix(true_classes, cnn_pred_classes)
sns.heatmap(cm_cnn, annot=True, fmt='d', cmap='Blues')
print(classification_report(true_classes, cnn_pred_classes,
target_names=class_names))
# Same steps repeated for MobileNetV2
print('Winner:', 'MobileNetV2' if mob_test_acc > cnn_test_acc else 'CNN')
```

What We Did:

We ran both saved models on the 436 unseen test images to get final performance metrics. evaluate() computes overall test accuracy and loss. predict() generates raw probability arrays for each image. np.argmax() converts probabilities to predicted class indices. confusion_matrix() creates a grid showing correct vs incorrect predictions per class — diagonal cells are correct predictions, off-diagonal cells are misclassifications. classification_report() computes precision (of all predicted Closed, how many are actually Closed), recall (of all actual Closed, how many were predicted Closed), and F1-score (harmonic mean of precision and recall) for each class individually. Finally both accuracies are compared to announce the winning model.

Input:

436 test images from split_dataset/test — never seen by either model during training or validation. True class labels from test_generator.classes.

Output:

CNN Test Accuracy: 80.73%, CNN Test Loss: printed. Confusion matrix heatmap in blue showing strong performance on Closed and Open (95% each) but weakness on yawn (52%). MobileNetV2 Test Accuracy: 92.20%, Confusion matrix heatmap in green showing near-perfect on Closed (100%) and Open (100%), strong on no_yawn (86%) and yawn (84%). Classification reports for both models printed. Final comparison: CNN 80.73% vs

MobileNetV2 92.20%. Winner announced: MobileNetV2 — with 11.47% higher accuracy.

Block 16 — Decision Fusion — Map 4 Classes to 3 Fatigue Levels

Code:

```
def get_fatigue_level(prediction):
class_names = ['Closed','Open','no_yawn','yawn']
predicted_class = class_names[np.argmax(prediction)]
if predicted_class in ['Open','no_yawn']:
return 0, 'Alert'
elif predicted_class == 'yawn':
return 1, 'Mild Fatigue'
else: # Closed
return 2, 'Severe Fatigue'
mob_preds = mobilenet_model.predict(test_generator)
fatigue_levels = [get_fatigue_level(p) for p in mob_preds]
plt.bar(fatigue_counts.keys(), fatigue_counts.values(),
color=['green','orange','red'])
```

What We Did:

We defined a decision fusion function that translates the model's 4-class technical output into a practical 3-level fatigue system. np.argmax() picks the class with the highest probability. The mapping logic reflects real-world drowsiness assessment: Open eyes and no yawning mean the driver is Alert (Level 0, safe to continue). Yawning indicates early-stage fatigue (Level 1 — Mild Fatigue, warning needed). Closed eyes indicate severe drowsiness (Level 2 — Severe Fatigue, emergency action required). This abstraction bridges the gap between machine learning output and practical automotive safety intervention.

Input:

MobileNetV2 prediction probability arrays for all 436 test images — each array contains 4 probability values (one per class) that sum to 1.0.

Output:

Fatigue level distribution printed: Alert (Level 0): 53.2% of test frames (232 images), Mild Fatigue (Level 1): 21.6% of test frames (94 images), Severe Fatigue (Level 2): 25.2% of test frames (110 images). A color-coded bar chart displayed with green bar (Alert), orange bar (Mild Fatigue), and red bar (Severe Fatigue) showing the distribution visually. Printed: 'Decision fusion done!'

Block 17 — Fatigue Progression Curve Over Simulated Driving Session

Code:

```
frames_per_minute = 10
num_minutes = len(session_fatigue) // frames_per_minute
```

```
for i in range(num_minutes):
avg_fatigue = np.mean(session_fatigue[i*10:(i+1)*10])
avg_fatigue_per_interval.append(avg_fatigue)
plt.axhspan(0, 0.5, color='green', label='Alert Zone')
plt.axhspan(0.5, 1.5, color='orange', label='Mild Fatigue Zone')
plt.axhspan(1.5, 2.0, color='red', label='Severe Fatigue Zone')
plt.axhline(y=0.5, color='orange', linestyle='--')
plt.axhline(y=1.5, color='red', linestyle='--')
plt.plot(time_intervals, avg_fatigue_per_interval)
```

What We Did:

We treated all 436 test predictions as frames from a continuous driving session. The predictions were grouped into 1-minute intervals assuming 10 frames per minute. For each interval, np.mean() calculated the average fatigue level. This produces a time-series progression curve showing how fatigue evolves over the simulated session. The chart includes colored background zones marking the three fatigue regions, dashed threshold lines at 0.5 and 1.5, and a blue line showing fatigue evolution. Code also identifies the exact transition minutes where the driver crosses from Alert to Mild, and from Mild to Severe Fatigue — critical information for timing real-world safety alerts.

Input:

All 436 MobileNetV2 test predictions converted to fatigue levels (0, 1, or 2) and treated as a sequential stream of driving frames, grouped into 43 one-minute intervals of 10 frames.

Output:

A line graph displayed showing driver fatigue progression across 43 simulated driving minutes. Green background zone (Alert: 0-0.5), orange zone (Mild Fatigue: 0.5-1.5), red zone (Severe: 1.5-2.0) clearly visible. Transition minutes printed: when Alert transitions to Mild Fatigue, and Mild to Severe Fatigue (or 'Driver stayed Alert/did not reach Severe'). Printed: Total session 43 minutes, Average fatigue level: 0.72 (in Mild range). Printed: 'Fatigue Progression Curve Complete!'

Block 18 — Final Project Summary and Accuracy Comparison Chart

Code:

```
print('CNN Test Accuracy: 80.73%')
print('MobileNetV2 Test Accuracy: 92.20%')
print('Alert (0): No action needed')
print('Mild Fatigue (1): Play warning sound')
print('Severe Fatigue (2): Emergency alert + Auto brake')
bars = plt.bar(['Custom CNN','MobileNetV2'], [80.73, 92.20],
color=['steelblue','green'])
for bar, acc in zip(bars, accuracies):
plt.text(bar center, bar height+1, f'{acc}%', ...)
plt.show()
print('Project Complete!')
```

What We Did:

The final block consolidates all results into a structured printed summary covering: dataset statistics (total images, train/val/test split, class names), model performance comparison (both CNN and MobileNetV2 test accuracies and class-wise strengths), decision fusion distribution (percentage of Alert, Mild, and Severe frames), fatigue progression findings (session length and average fatigue), and business recommendations mapping each fatigue level to a specific real-world automotive action. A final bar chart visually compares both model accuracies with percentage labels on top of each bar.

Input:

All computed results from every previous block — model accuracies, fatigue distributions, session statistics, and class-wise performance metrics.

Output:

Complete structured project summary printed on screen. Final accuracy comparison bar chart displayed showing: Custom CNN bar (steelblue) at 80.73% and MobileNetV2 bar (green) at 92.20% with percentage labels on each bar — MobileNetV2 clearly taller. Business recommendations clearly stated: Level 0 Alert = no action, Level 1 Mild Fatigue = play warning sound, Level 2 Severe Fatigue = emergency alert plus automatic braking. Printed: 'Project Complete!' confirming successful end-to-end execution of the entire Driver Drowsiness Detection pipeline.

Final Results Summary

Item	Details
Total Dataset	2,900 images — Closed, Open, no_yawn, yawn (balanced, ~725 each)
Data Split	Train: 2029 images | Validation: 435 images | Test: 436 images
Augmentation	Rotation, Zoom, Flip, Brightness on training data only
Custom CNN Accuracy	80.73% — Strong on Closed/Open (95%), weak on yawn (52%)
MobileNetV2 Accuracy	92.20% — Perfect on Closed/Open (100%), strong on yawn (84%)
Winner	**MobileNetV2 — 11.47% higher accuracy than custom CNN**
Fatigue Levels	Level 0: Alert (53.2%) | Level 1: Mild (21.6%) | Level 2: Severe (25.2%)
Driving Simulation	43 minutes | Average fatigue: 0.72 (Mild range)
Safety Actions	Alert: none | Mild: warning sound | Severe: emergency alert + auto brake
Platform	Google Colab with TensorFlow, Keras, NumPy, Matplotlib, Seaborn, Sklearn

---

This document covers all 18 code blocks of the Driver Drowsiness Detection project — each explained with the exact code written, what each line does, what was given as input, and what was received as output — providing a complete reference for project review and presentation.